

Table of Contents

1.Abstract:	2
2.Introduction:	2
3.Methodology:	4
4.Implementation:	7
5.Results & Analysis:	9
6.Discussion:	12
7.Conclusion:	13

PROJECT REPORT

2D PHYSICS-BASED PUZZLE GAME USING MATTER.JS & HTML5 CANVAS

1. Abstract

This project focuses on developing an interactive, 2D physics-based puzzle game using JavaScript, the Matter.js physics engine and the HTML5 Canvas API. The main objective is to build a real-time browser application in which the player launches a projectile from a slingshot to topple stacked structures and knock all target blocks below a defined altitude, while managing a limited number of shots. The system renders a dynamic world containing static ground and walls, stackable rigid bodies and a drag-and-release slingshot. The methods implemented include a central game and physics loop built on the Matter.js engine, runner and world contexts, Mouse Constraint based input handling, impulse-based projectile launching, collision filtering for target blocks, a win/loss state system, JSON-driven level definitions and progress persistence using the LocalStorage API. The results show that the application reliably simulates stable block structures, converts drag vectors into realistic launch forces, tracks destruction metrics, and manages level progression across multiple maps. This project demonstrates the practical application of rigid-body physics simulation, event-driven programming and data-driven level design in a web environment.

2. Introduction

-Background

JavaScript is the primary language of the web and, together with the HTML5 Canvas element, allows developers to build rich 2D interactive applications that run in any modern browser without installing additional software. Physics-based puzzle games are a popular genre in which player skill is expressed through the manipulation of forces, mass, friction and collisions rather than scripted animation. Matter.js is a lightweight 2D rigid-body physics engine for the web that provides bodies, constraints, collision detection, a runner and mouse interaction, making it well suited for learning how real-time physics simulation is integrated with a rendering and input pipeline.

-Problem Statement

Concepts such as rigid-body dynamics, friction, restitution, impulses and constraint-based interaction can be difficult to understand when studied only in theory. Developers also face practical challenges in keeping a physics world stable, mapping engine coordinates to a rendered viewport, handling real-time input and structuring game state and levels in a maintainable way. Constructing an interactive physics puzzle game addresses these challenges by applying them to a complete, playable product.

-Objectives

The objectives of this project are:

- To design and implement a 2D physics-based puzzle game using Matter.js and HTML5 Canvas.
- To build a central game and physics loop integrating the Matter.js engine, runner and world.
- To create a stable static environment and dynamic, stackable block structures by tuning density, weight, friction and restitution.
- To implement a slingshot mechanic using Mouse Constraint, drag-and-release vector calculations and impulse-based projectile launching.
- To implement target blocks with separate collision filters, destruction tracking and off-screen removal.
- To build a win/loss state system with "Level Complete" and "Retry" overlays.
- To design a JSON-driven level system with 3 to 5 maps of progressive difficulty and persist unlocked progress using LocalStorage.

-Scope

The scope of this project includes a complete 2D web game developed with JavaScript, HTML5 Canvas and Matter.js. The application features a slingshot with multi-shot tracking, destructible block structures, target tracking, win and loss states, multiple JSON-defined levels, local progress saving, particle dust trails and block fragmentation effects. The project runs locally in the browser and does not include network multiplayer functionality or server-side database persistence.

3. Methodology

-Approach

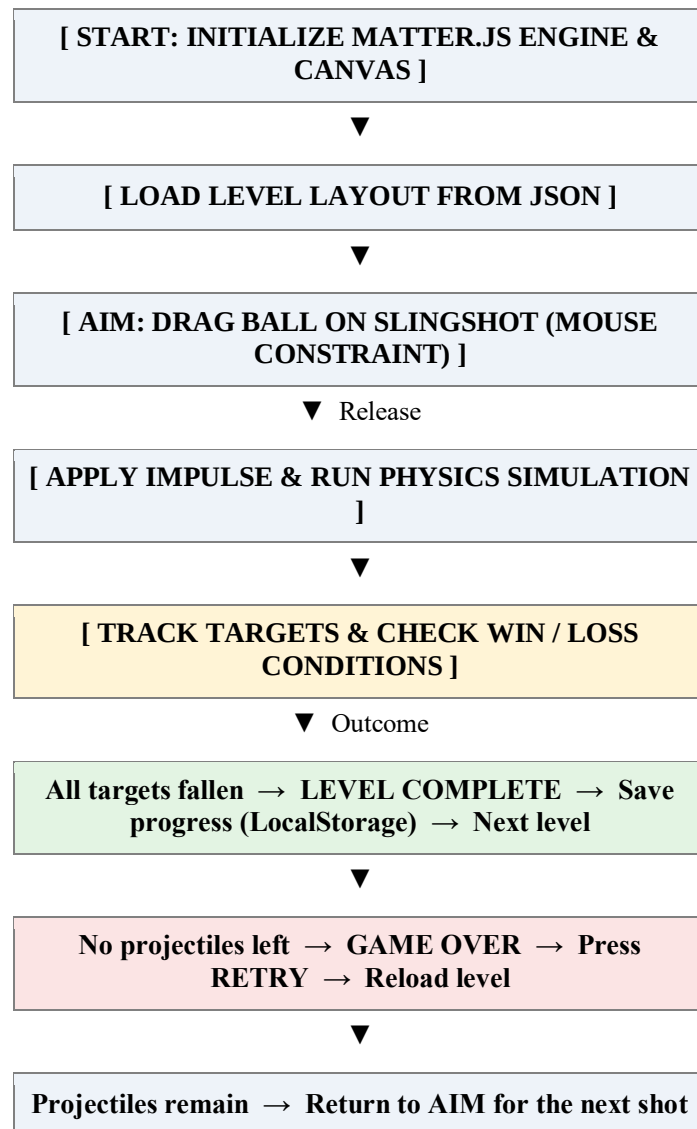
The project follows an iterative, event-driven and modular approach organized into a ten-week workplan. The Matter.js runner advances the physics engine at a fixed time step, while the game loop reads the updated body positions, tracks targets, evaluates win and loss conditions and redraws the Canvas on every frame. Each week adds one self-contained capability, from the environment and structures to the slingshot, launch forces, game states, levels, persistence and final quality assurance, so that every phase produces a testable deliverable.

-Algorithm

Algorithm for the 2D Physics-Based Puzzle Game Loop and Level Logic:

1. Start the application, create the Canvas viewport and initialize the Matter.js engine, runner and world.
2. Create the static environment (ground, boundaries and rigid walls) with configured friction and restitution values.
3. Load the selected level from its JSON definition, clearing the previous world and constructing the stackable structures and target blocks at the specified coordinates.
4. Create the projectile ball and bind it to the slingshot anchor position. Enable Mouse Constraint input so the player can drag the ball.
5. On mouse release, compute the drag vector from the anchor, apply the snap-back elastic logic and apply the corresponding impulse/force vector to the projectile body. Decrease the remaining projectile count.
6. During each engine update, track positions, velocities and height coordinates of the projectile and target blocks, and remove bodies that move off-screen or out of bounds.
7. Evaluate the game state: if all target blocks have fallen below the altitude threshold, show the "Level Complete" overlay, save the unlocked level to LocalStorage and allow the next level to load.
8. If the player runs out of projectiles while targets remain, show the "Retry" overlay and reload the current level on request.
9. Render the frame to the Canvas, including structures, slingshot, particle dust trails and block fragmentation effects, and repeat from step 4 for the next shot.

-Flowchart



-Tools and Technologies

- Programming Language: JavaScript (ES6), HTML5, CSS3
- Physics Engine: Matter.js (Engine, Runner, World, Bodies, Constraint, MouseConstraint)
- Graphics: HTML5 Canvas 2D API
- Data & Storage: JSON level definitions and the browser LocalStorage API
- Development Environment: Visual Studio Code and Google Chrome (with developer tools)
- Hardware: Laptop

-Experimental Design

The system is verified by executing repeated gameplay runs across different launch angles, drag distances and level layouts. Tests verify structural stability of stacked blocks at rest, responsiveness of the Mouse Constraint slingshot, correctness of the impulse applied to the projectile, accuracy of target tracking and off-screen removal, and the correct triggering of the win and loss states. JSON levels are tested by loading every map in sequence, and persistence is tested by closing and reopening the browser to confirm that unlocked levels are restored. Frame rate is monitored with the browser developer tools to confirm stable performance, and extreme-angle and high-force launches are used to detect collision tunnelling or unstable stacks.

4. Implementation

-System Architecture

The application architecture is divided into four core functional modules:

1. **Input & Control Module:** Uses Matter.js Mouse Constraint and drag-and-release vector calculations to let the player pull back and release the projectile, and handles menu and retry interactions.
2. **Physics Module:** Uses the Matter.js engine, runner and world to simulate static and dynamic bodies, apply impulses, manage friction and restitution, and apply collision filters to target blocks.
3. **Game Logic & State Module:** Tracks target positions and heights, remaining projectiles, win and loss conditions, level progression and overlay states.
4. **Rendering & Data Module:** Draws the world on the HTML5 Canvas (including a debug renderer view), applies particle effects, parses JSON level files and saves progress to LocalStorage.

-Components

- **Game/Physics Loop Controller:** Central loop that synchronizes the Matter.js runner with Canvas rendering.
- **Static Environment Builder:** Creates the ground, boundaries and rigid walls with configured friction and restitution, and maps engine units to the viewport.
- **Structure Builder:** Generates stackable rectangles and circles with tuned density, weight and friction for stable balancing.
- **Slingshot Controller:** Binds the projectile to the anchor, tracks the drag vector, performs the elastic snap-back and applies the launch impulse.
- **Target & Collision Manager:** Defines target blocks with separate collision filters, tracks positions, velocities and heights, and removes off-screen bodies.
- **State Manager:** Implements win and loss criteria and the "Level Complete" and "Retry" overlays.
- **Level Manager:** Parses JSON schemas to clear and construct each level dynamically and sequences the 3 to 5 maps.
- **Persistence & Effects Module:** Stores unlocked level progression in LocalStorage and renders particle dust trails and block fragmentation.

-Level Data System

Levels are described by structured JSON schemas containing the coordinates, type, size and material properties of every block, the position of the targets, the projectile count and the slingshot anchor. A level parser clears the existing Matter.js world and rebuilds the scene from the selected configuration, so new maps of progressively increasing difficulty can be added by editing data rather than code. The index of the highest unlocked level is written to the LocalStorage API so that player progression persists between sessions.

-Weekly Workflow

Week	Development Phase	Key Implementation Focus	Key Deliverable
Week 1	Environment & Architecture Setup	Web project framework, Canvas viewport, Matter.js engine/runner/world, central game loop	Functional game loop & empty canvas template
Week 2	Environment & Static Obstacles	Static ground, boundaries and walls; friction and restitution; engine-to-viewport scale mapping	Static environment with rigid boundaries
Week 3	Dynamic Structure Building	Stackable rectangles and circles; density, weight and friction tuning; debug renderer	Destructible / stackable block structures
Week 4	Sling Mechanics & Controls	Mouse Constraint controls; drag-and-release vector tracking; ball bound to slingshot anchor	Interactive drag-and-release slingshot
Week 5	Projectile Launch & Forces	Snap-back elastic release; impulse/force vector on release; multi-shot tracking	Fully launchable projectile with realistic impulse
Week 6	Target Blocks & Logic	Target blocks with separate collision filters; position/velocity/height tracking; off-screen removal	Target tracking & destruction metrics
Week 7	Win/Loss & Level State	Win when all targets fall below altitude threshold; loss when projectiles run out; overlays	State system with dynamic game menu overlays
Week 8	JSON Level Design	JSON layout schema; 3 to 5 maps with progressive difficulty; dynamic level parser	JSON-driven multi-level sequence manager
Week 9	Local Save & Polish	LocalStorage level progression; graphics, dust trails, block fragmentation; rendering optimization	Polished game loop with local progression data
Week 10	QA, Bug Fixing & Delivery	Collision and extreme-angle testing; balance tuning; final bundle and asset optimization	Production-ready web game package

5. Results & Analysis

-Result

The application successfully executes a 2D physics puzzle environment with a stable static world, balanced block structures, a responsive drag-and-release slingshot and realistic projectile impulses. Testing confirmed that stacked structures remain at rest until struck, that the projectile follows a physically plausible trajectory after release, and that target blocks falling below the altitude threshold are correctly counted. The "Level Complete" overlay appears when all targets have fallen, the "Retry" overlay appears when the player runs out of projectiles, and the unlocked level progression is restored from LocalStorage after the page is reloaded.

-Analysis

The results confirm that Matter.js, combined with the HTML5 Canvas, is capable of handling real-time 2D rigid-body simulation in the browser. Careful tuning of density, weight and friction was essential for stable stacking, and the use of separate collision filters allowed targets to be distinguished from ordinary blocks without additional bookkeeping. The JSON-driven level system separated game content from game logic, making it straightforward to extend the game with new maps, and the optimized rendering pipeline kept the frame rate stable during particle effects and block fragmentation.

-Code (Output)

The following figures show the output of the game at the key stages of development. The screenshots illustrate the static environment, the stackable structures, the slingshot interaction, the projectile launch, the target tracking view, the win and loss overlays, the level sequence and the final polished gameplay. The visual output is given in next page :



Figure1: The 'Start Game' interface with game Title.

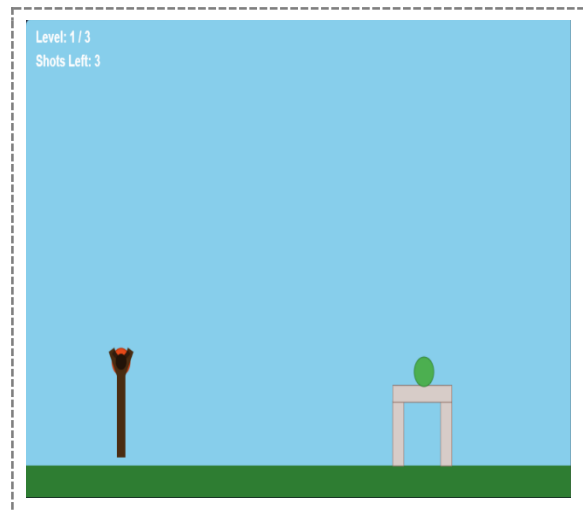


Figure2: First and basic 'level' interface.

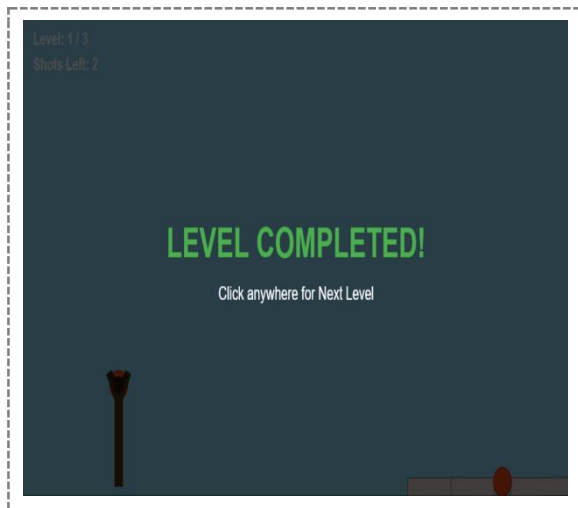


Figure3: Level completion and asking for 'Next level'.

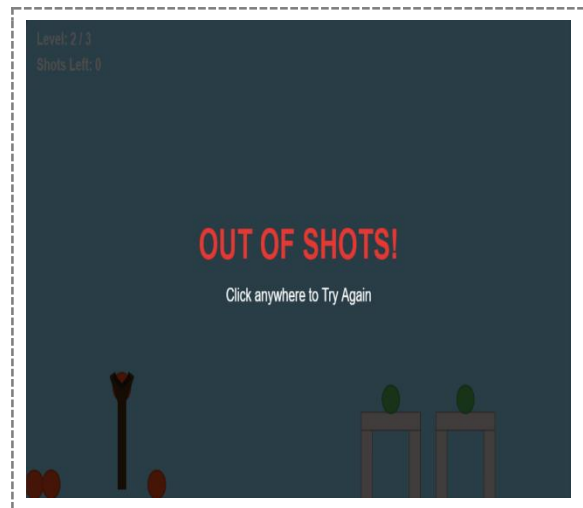


Figure4: Level failed interface with dialogue 'Out of Shots'.

6. Discussion

-Interpretation

The implementation results demonstrate that Matter.js and the HTML5 Canvas are fully capable of supporting a complete physics-based puzzle game, including structure stability, force-based interaction, state management and persistent progression. Separating the physics, input, game logic and rendering responsibilities simplified debugging and allowed each weekly deliverable to be tested independently. The system meets its target goals: a stable physical environment, an intuitive slingshot control, reliable win and loss detection and data-driven levels.

-Key Technical Achievements

- **Stable Physics Environment:** Tuning density, weight, friction and restitution, together with a debug renderer for verifying alignment, produced structures that balance reliably and respond realistically to impact.
- **Slingshot and Impulse Mechanics:** Combining Mouse Constraint input with drag-vector calculations and an elastic snap-back allowed drag distance and direction to be converted into a realistic launch impulse, with multi-shot tracking.
- **Collision Filtering and Target Tracking:** Separate collision filters for target blocks, together with position, velocity and height tracking, made win detection accurate and kept the simulation clean by removing off-screen bodies.
- **Data-Driven Levels and Persistence:** JSON level schemas and a dynamic level parser allowed multiple maps to be created and sequenced easily, while LocalStorage preserved player progress between sessions.

-Challenges

- **Structural Stability:** Stacks of blocks could jitter or collapse unexpectedly, which was resolved by adjusting density, friction and mass ratios.
- **Extreme-Angle Collisions:** Very fast or extreme-angle launches could cause unstable collisions, which required dedicated testing and stability fixes in the final week.
- **Coordinate Scale Mapping:** Keeping the Matter.js world and the Canvas viewport aligned required a clear scale mapping and a debug renderer for verification.
- **Rendering Performance:** Particle trails and block fragmentation increased rendering cost, so the pipeline was optimized to maintain a stable, high frame rate.

-Limitations & Potential Enhancements

While the project meets all functional requirements, future iterations could expand the gameplay experience:

1. **Additional Projectile and Block Types:** Introducing projectiles with different masses and special behaviours, and blocks made of different materials such as glass, wood and stone.

2. Scoring and Star Ratings: Awarding scores or star ratings based on remaining shots and destruction achieved.
3. Online Leaderboard: Replacing local-only persistence with a server-side database so that scores can be shared between players.
4. Mobile and Touch Support: Adapting the slingshot controls and layout for touch screens and smaller viewports.

7. Conclusion

-Summary

The 2D Physics-Based Puzzle Game project successfully demonstrates how rigid-body physics, event-driven input handling, state management and data-driven design can be combined to build an engaging browser game. By building the project incrementally, from the Canvas and Matter.js setup in Week 1 through structures, the slingshot, launch forces, targets, game states, JSON levels and local saving to final quality assurance in Week 10, the software architecture remains clean, maintainable and robust. The resulting application provides responsive controls, realistic physical behaviour, clear win and loss feedback and persistent progression, meeting all target objectives.

-Contributions

The project demonstrates how the Matter.js physics engine and native web technologies can build an interactive puzzle game without a full game engine. Key contributions include establishing a central game and physics loop, building stable stackable structures, implementing a Mouse Constraint slingshot with impulse-based launching, applying collision filters for target tracking, implementing a win/loss state system, creating a JSON-driven multi-level manager and persisting progress with LocalStorage. This project serves as an extensible foundation for learning web-based physics simulation and 2D game development.